

Recomendaciones de Librerías y Frameworks

Documento de referencia para el equipo de desarrollo. Las recomendaciones están alineadas con el stack definido en [ADR-3](#) (Vue.js + Python/FastAPI + Node.js + Supabase) y con las restricciones de presupuesto, conectividad rural y soporte PWA establecidas en los ADRs del proyecto.

Stack base confirmado (ADR-3)

Capa	Tecnología definida
Frontend	Vue.js 3 + Vite
Mapas	Leaflet + OpenStreetMap
Backend (servicios de dominio)	Python + FastAPI
API Gateway	Node.js + Express / Fastify
Base de datos	Supabase (PostgreSQL + PostGIS)
Autenticación	Supabase Auth + JWT

Frontend

Consideración previa: dos UIs con naturalezas distintas

El proyecto tiene dos aplicaciones frontend con requisitos diferentes:

- **PWA ciudadana:** orientada a móvil, debe cargar rápido en redes rurales 3G/4G, necesita acceso a GPS, cámara y soporte offline. Bundle máximo ~200 KB (ADR-3).
- **Plataforma institucional / admin:** orientada a escritorio, necesita tablas con filtros y paginación, gráficos del dashboard y formularios complejos de gestión.

Esta diferencia justifica recomendar librerías distintas para cada contexto en lugar de una solución única.

Librería de componentes — PWA ciudadana

Recomendación: Quasar Framework

```
npm create quasar
```

Quasar es un framework Vue 3 que incluye en su CLI soporte nativo para PWA (service workers, manifest, modo offline), acceso a cámara y GPS vía Capacitor/Cordova, y notificaciones push. Esto resuelve directamente el requerimiento de la segunda entrega (estrategia offline, ADR-5) sin instalar dependencias adicionales.

Sus componentes están optimizados para mobile-first y el bundle base compilado es ~150 KB gzipped, dentro del umbral del ADR-3. También cubre desktop, por lo que si en el futuro se decide unificar las dos PWAs en un único proyecto, Quasar escala sin cambio de librería.

Alternativa si se prefiere algo más liviano: Naive UI — componentes Vue 3 TypeScript-first, sin opinión sobre el layout.

Librería de componentes — Plataforma institucional / admin

Recomendación: PrimeVue

```
npm install primevue
```

PrimeVue tiene el componente `DataTable` más completo del ecosistema Vue 3: filtros por columna, ordenamiento, paginación, exportación a CSV y PDF nativa, y selección múltiple. Es exactamente lo que necesitan las pantallas de Gestión de Reportes (CU-09, CU-10, CU-11) y la exportación del Analytics Service (CU-13).

Adicionalmente, su componente `Chart` envuelve Chart.js, cubriendo los gráficos de barras y anillo del dashboard institucional sin instalar una segunda librería. Es MIT y tree-shakeable: solo entra al bundle lo que se importa.

Alternativa: Vuetify — también cubre el caso, pero su bundle base es mayor y tiene más opinión sobre el diseño visual (Material Design).

Estado global

Recomendación: Pinia + `pinia-plugin-persistedstate`

```
npm install pinia pinia-plugin-persistedstate
```

Pinia es el gestor de estado oficial de Vue 3 (sucesor de Vuex). Liviano, con soporte TypeScript excelente y DevTools integradas. El plugin de persistencia permite cachear en `localStorage` los catálogos que cambian poco (tipos de incidente, especies, protocolos), reduciendo las llamadas al backend en cada sesión y mejorando la experiencia en conexiones lentas.

Justificación: Sin persistencia, cada vez que el usuario abre el formulario de reporte se hacen dos llamadas al backend (tipos de incidente + especies). Con Pinia persistido, esas llamadas solo ocurren si el caché expiró.

PWA y service workers

Recomendación: vite-plugin-pwa + Workbox

```
npm install -D vite-plugin-pwa
```

Genera el service worker automáticamente desde la configuración de Vite. La estrategia recomendada por recurso:

Recurso	Estrategia Workbox	Justificación
Tiles del mapa	CacheFirst	Cambian poco, costosos de recargar
Catálogos (tipos, especies)	CacheFirst (TTL: 24h)	Datos estables
Reportes del mapa	NetworkFirst	Deben estar actualizados
Formulario de creación	NetworkFirst + cola offline	Dato crítico, no se puede perder

La cola offline (Workbox BackgroundSync) es lo que implementa el escenario de disponibilidad del ADR: reportes guardados localmente que se sincronizan automáticamente al recuperar conexión.

Mapas

Recomendación: vue3-leaflet (vue-leaflet)

```
npm install @vue-leaflet/vue-leaflet leaflet
```

Wrapper oficial de Leaflet para Vue 3. Expone los componentes como <l-map> , <l-tile-layer> , <l-marker> y <l-popup> con props y eventos Vue nativos. Alternativa a instanciar Leaflet directamente en onMounted , lo que resulta en código más limpio y reactivo.

```
<l-map :zoom="13" :center="[latitude, longitude]">
  <l-tile-layer url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png" />
  <l-marker v-for="report in reports" :lat-lng="[report.latitude, report.longitude]" />
</l-map>
```

Gráficos (dashboard)

Recomendación: vue-echarts (Apache ECharts)

```
npm install vue-echarts echarts
```

ECharts tiene mejor rendimiento que Chart.js en datasets con muchos puntos (relevante cuando el dashboard admin muestra distribución territorial de cientos de reportes). El componente `vue-echarts` es el wrapper oficial para Vue 3.

Cubre todos los gráficos del Analytics Service: barras apiladas (distribución por tipo de incidente), anillo/donut (análisis de estados), y barras horizontales (top comunas y especies).

Alternativa más simple para el MVP: `vue-chartjs` + Chart.js. Más liviano, más fácil de configurar, suficiente para el dashboard institucional mínimo definido en el acta del 15/06.

Formularios y validación

Recomendación: VeeValidate 4 + Zod

```
npm install vee-validate zod
```

VeeValidate v4 se integra con Vue 3 Composition API mediante `useForm` y `useField`. Zod define los schemas de validación de forma type-safe. El formulario de reporte debe completarse en menos de 20 segundos (escenario de usabilidad), por lo que la validación en tiempo real es crítica para no interrumpir el flujo del usuario.

```
// Ejemplo: schema del formulario de reporte
const reportSchema = z.object({
  incidentTypeId: z.number({ required_error: 'Selecciona un tipo de incidente' }),
  animalCount: z.number().min(1).max(99),
  description: z.string().min(10).max(500),
  latitude: z.number(),
  longitude: z.number(),
  speciesId: z.number().optional(),
  reportPhotoUrl: z.string().url().optional(),
});
```

Cliente HTTP

Recomendación: Axios

```
npm install axios
```

Estándar del ecosistema. Se configura un interceptor global para adjuntar el JWT de Supabase Auth en cada request (`Authorization: Bearer <token>`) y otro interceptor de respuesta para capturar los 401 y redirigir al login automáticamente.

```
axios.interceptors.request.use(config => {
  const { data: { session } } = supabase.auth.getSession();
  if (session) config.headers.Authorization = `Bearer ${session.access_token}`;
  return config;
});
```

Backend — Python + FastAPI

FastAPI incluye Pydantic v2 y generación automática de Swagger/OpenAPI, cubriendo el requisito de documentación de contratos del ADR-3. Las recomendaciones son para la capa de base de datos y comunicación entre servicios.

ORM y acceso a PostGIS

Recomendación: SQLAlchemy 2 + GeoAlchemy2

```
pip install sqlalchemy geoalchemy2 asyncpg
```

SQLAlchemy 2 con soporte async es el stack estándar para FastAPI. GeoAlchemy2 agrega los tipos geoespaciales de PostGIS (`Geometry` , `Geography`) directamente en los modelos Python, permitiendo escribir consultas espaciales (`ST_DWithin` , `ST_Within` , `ST_Distance`) sin SQL en texto plano.

Crítico para el Institution Service, cuyas derivaciones usan consultas de jurisdicción territorial sobre PostGIS.

```
from geoalchemy2 import Geography
from sqlalchemy.orm import mapped_column

class Institution(Base):
    __tablename__ = "institutions"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str]
    location: Mapped[Geography] = mapped_column(Geography(geometry_type="POINT", srid=4326))
```

Driver de base de datos: `asyncpg` — driver async nativo de PostgreSQL, el más rápido disponible para Python. SQLAlchemy 2 lo usa como backend con la URL `postgresql+asyncpg://`.

Validación de JWT (servicios de dominio)

Recomendación: `python-jose` + `cryptography`

```
pip install python-jose[cryptography]
```

Cada servicio de dominio debe validar el JWT emitido por Supabase Auth antes de procesar cualquier request. `python-jose` verifica la firma contra la clave pública de Supabase sin hacer una llamada extra al Identity Service en cada request, lo que reduce la latencia y el acoplamiento.

```
from jose import jwt, JWTError

def verify_token(token: str) -> dict:
    try:
        payload = jwt.decode(token, SUPABASE_JWT_SECRET, algorithms=["HS256"])
```

```
        return payload
    except JWTErrror:
        raise HTTPException(status_code=401, detail="Token inválido")
```

Llamadas entre servicios

Recomendación: HTTPX (modo async)

```
pip install httpx
```

Cliente HTTP async para Python, la recomendación oficial de FastAPI para llamadas service-to-service. Tiene la misma API que `requests` pero no bloquea el event loop, lo que es crítico para el Sighting Service que hace tres llamadas encadenadas (Map → Institution → Content) durante la creación de un reporte.

```
async with httpx.AsyncClient() as client:
    validation = await client.post(f"{MAP_SERVICE_URL}/validate-coordinates", json=coords)
    derivation = await client.post(f"{INSTITUTION_SERVICE_URL}/institutions/derive", json=payload)
```

Storage de imágenes

Recomendación: `supabase-py`

```
pip install supabase
```

Para gestionar las subidas de fotos de reportes y POIs al Storage de Supabase desde el backend. Evita que el frontend tenga acceso directo al bucket y centraliza el control de acceso por rol.

Exportación CSV/PDF (Analytics Service)

Recomendación: `pandas` + `reportlab`

```
pip install pandas reportlab openpyxl
```

`pandas` para generar los DataFrames y exportar a CSV/Excel. `reportlab` para generar los PDF del dashboard. Implementa el patrón Strategy definido en los ADRs: cada formato es una clase separada con la misma interfaz `generate(data) -> bytes`.

API Gateway — Node.js

Framework

Recomendación: Fastify (ya definido en ADR-3 como preferido)

```
npm install fastify
```

Fastify tiene mayor throughput que Express (~35.000 req/s vs ~15.000 req/s en benchmarks estándar), adecuado para el punto de entrada único que recibe todas las solicitudes de ambas PWAs.

Plugins recomendados

Plugin	Propósito
@fastify/jwt	Valida y decodifica el JWT de Supabase en un hook global. Ningún handler necesita lógica de auth.
@fastify/http-proxy	Redirige el tráfico a cada servicio interno sin escribir handlers manuales por ruta.
@fastify/rate-limit	Limitación de tasa por IP. Protege los endpoints públicos de la PWA ciudadana.
@fastify/cors	CORS configurado por ambiente (desarrollo vs producción).
@fastify/helmet	Headers de seguridad HTTP (CSP, HSTS, X-Frame-Options).

```
// Ejemplo: hook global de autenticación
fastify.addHook('onRequest', async (request, reply) => {
  try {
    await request.jwtVerify();
  } catch (err) {
    reply.code(401).send({ error: 'You are unauthorized to make this request.' });
  }
});
```

Resumen ejecutivo

Capa	Librería recomendada	Alternativa
PWA ciudadana	Quasar Framework	Naive UI
Plataforma admin	PrimeVue	Vuetify

Estado global	Pinia + pinia-plugin-persistedstate	—
PWA / Offline	vite-plugin-pwa + Workbox	—
Mapas (Vue)	vue3-leaflet	—
Gráficos	vue-echarts	vue-chartjs
Formularios	VeeValidate 4 + Zod	—
HTTP client (frontend)	Axios	—
ORM + PostGIS	SQLAlchemy 2 + GeoAlchemy2	—
Driver PostgreSQL	asyncpg	psycopg3
JWT (backend)	python-jose	PyJWT
HTTP entre servicios	HTTPX	aiohttp
Storage	supabase-py	—
Exportación	pandas + reportlab	—
Gateway framework	Fastify	—
Gateway auth	@fastify/jwt	—
Gateway proxy	@fastify/http-proxy	—

Notas

- Todas las librerías recomendadas son de licencia MIT o Apache 2.0, sin costo de uso.
- No se recomienda React ni ninguna de sus librerías de componentes (shadcn/ui, MUI, Ant Design) en cumplimiento de la restricción del ADR-3.
- El uso de Quasar en la PWA ciudadana y PrimeVue en la plataforma admin implica dos proyectos Vue separados. Si el equipo prefiere un monorepo, ambas pueden coexistir bajo un workspace de `pnpm` o `Turborepo`.